

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №7

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Обход графа в глубину"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение обхода графа в глубину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. *Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

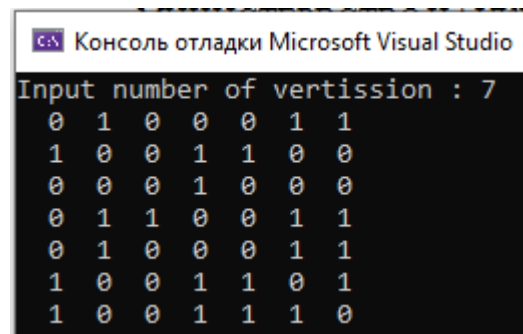
Пояснительный текст к программам

Эта программа реализует алгоритм обхода графа в глубину (DFS). Пользователь задаёт количество вершин графа, после чего программа автоматически генерирует симметричную матрицу смежности, где:

- 0 означает отсутствие ребра между вершинами
- 1 означает наличие ребра

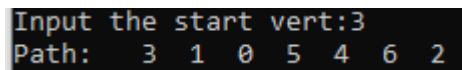
Главная диагональ матрицы заполняется нулями (исключаются петли). После отображения матрицы пользователь выбирает стартовую вершину, и программа выполняет обход в глубину, выводя порядок посещения вершин. По завершении вся выделенная память освобождается.

Результат программы



```
Консоль отладки Microsoft Visual Studio
Input number of vertission : 7
0 1 0 0 0 1 1
1 0 0 1 1 0 0
0 0 0 1 0 0 0
0 1 1 0 0 1 1
0 1 0 0 0 1 1
1 0 0 1 1 0 1
1 0 0 1 1 1 0
```

Рисунок 1 – вывод матрицу смежности размером 7



```
Input the start vert:3
Path: 3 1 0 5 4 6 2
```

Рисунок 2 – вывод начальную вершину 3
и путь обхода в глубину

Псевдокод

ПРОГРАММА ОБХОДА ГРАФА В ГЛУБИНУ

НАЧАЛО

ПРОГРАММА DFS(G, numG, visited, s)

visited[s] = 1

ВЫВЕСТИ s

ДЛЯ i ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

ЕСЛИ (G[s][i] == 1) И (visited[i] == 0) ТО

DFS(G, numG, visited, i)

КОНЕЦ ЕСЛИ

КОНЕЦ ЦИКЛА

КОНЕЦ ПРОГРАММЫ

ОСНОВНАЯ ПРОГРАММА

G — матрица смежности (двумерный массив)

numG — количество вершин в графе

current — стартовая вершина для обхода

visited — массив флагов пройденности вершин

ВЫВЕСТИ "Input number of vertices: "

ВВЕСТИ numG

visited = выделить_память(numG * размер_int)

G = выделить_память(numG * размер_указателя_на_int)

ДЛЯ i ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

G[i] = выделить_память(numG * размер_int)

КОНЕЦ ЦИКЛА

инициализировать_генератор_случайных_чисел(текущее_время)

ДЛЯ i ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

visited[i] = 0

ДЛЯ j ОТ i ДО numG-1 ВЫПОЛНИТЬ

ЕСЛИ i == j ТО

G[i][j] = 0

ИНАЧЕ

G[i][j] = G[j][i] = случайное_число(0, 1) // 0 или 1

КОНЕЦ ЕСЛИ

КОНЕЦ ЦИКЛА

КОНЕЦ ЦИКЛА

ДЛЯ i ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

ДЛЯ j ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

ВЫВЕСТИ G[i][j]

КОНЕЦ ЦИКЛА

ВЫВЕСТИ новую_строку

КОНЕЦ ЦИКЛА

ВЫВЕСТИ "Input the start vertex: "

ВВЕСТИ current

ВЫВЕСТИ "Path: "

DFS(G, numG, visited, current)

освободить_память(visited)

ДЛЯ i ОТ 0 ДО numG-1 ВЫПОЛНИТЬ

освободить_память(G[i])

КОНЕЦ ЦИКЛА

освободить_память(G)

КОНЕЦ ПРОГРАММЫ

ВЫПОЛНИТЬ ОСНОВНУЮ ПРОГРАММУ

КОНЕЦ

Визуализация

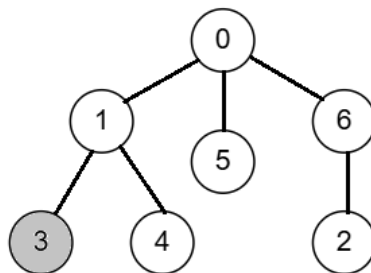


Рисунок 1 – вызов DFS(3)

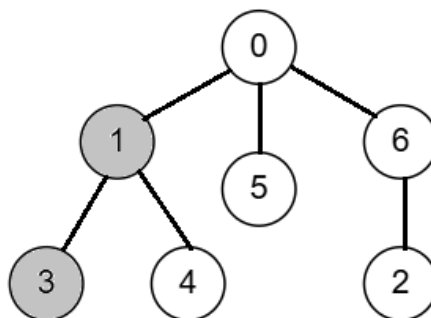


Рисунок 2 – вызов DFS(1)

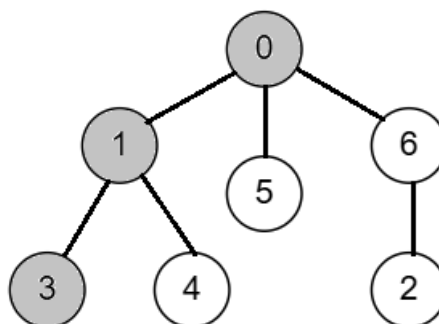


Рисунок 3 – вызов DFS(0)

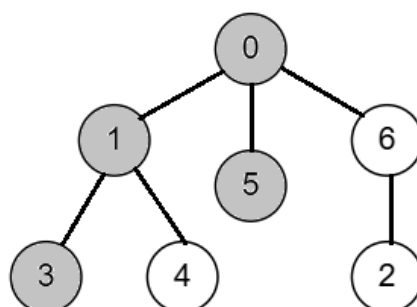


Рисунок 4 – вызов DFS(5)

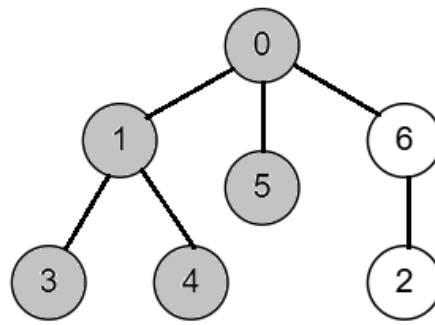


Рисунок 5 – вызов DFS(4)

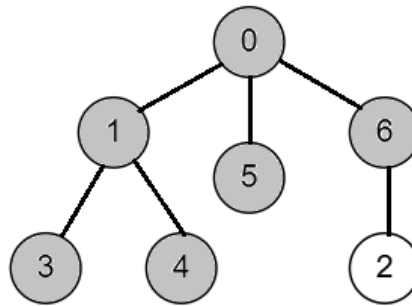


Рисунок 6 – вызов DFS(6)

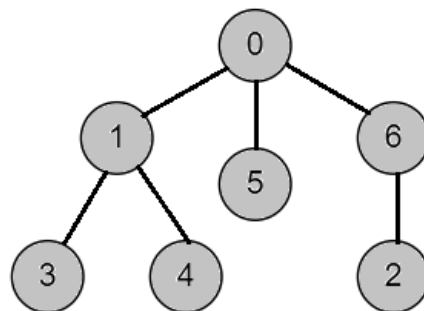


Рисунок 7 – вызов DFS(2)

Листинг

Файл 1- laba7.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <cstdlib>
#include <ctime>

void DFS(int** G, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d",s);

    for (int i = 0; i, numG; i++) {
        if (G[s][i] == 1 && visited[i] == 0)
            DFS(G, numG, visited, i);
    }
}

int main() {
    int** G;
    int numG;
    int current;
    int* visited;

    printf("Input number of vertission : ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++)
        G[i] = (int*)malloc(numG * sizeof(int));

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    printf("Input the start vert:");
    scanf("%d", &current);

    printf("Path: ");

    DFS(G, numG, visited, current);

    free(visited);
    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);

    return 0;
}
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №7. В процессе выполнения работы были использованы знания обхода графов в глубину.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>